

Food Store

Sistema de Gestión de Pedidos de Comida

Frontend Web (TypeScript + Vite) · Backend Java (JPA / Hibernate / H2)
Menú de Consola + API REST con Spring Boot

Comisión 15

Valletto, Marianela <maruvalletto@gmail.com>

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional

Programación III

Coordinador: Enferrel, Ariel

Profesores: Cortez, Alberto

Rigoni, Cinthia

30/06/2026

ÍNDICE

Introducción	3
Objetivos	3
Tecnologías utilizadas	3
Recursos adicionales	4
Arquitectura y flujo de funcionamiento	4
Flujo general de la aplicación	5
Frontend Web	5
Estructura y módulos	5
Capa de integración (api.ts)	6
Backend: modelo de dominio	7
Entidades	7
Relaciones entre entidades	7
Enums	8
Backend — Modo consola (JPA puro)	8
Alta de pedido (transacción atómica)	9
El alta de pedidos constituye la operación de negocio más importante del sistema.	9
Backend — API REST (Spring Boot)	9
Decisiones técnicas	11
Datos de prueba y credenciales	11
Credenciales de prueba	12
Reglas de negocio implementadas	13
Cobertura de Historias de Usuario	13
Backend	13
Frontend	14
Instrucciones de ejecución	14
Backend — API REST (Spring Boot)	14
Backend — Menú de consola (JPA)	14
Frontend	15
Ejecución integrada	15
Conclusión	16

Introducción

El presente informe documenta el Trabajo Práctico Integrador (TPI) de la materia Programación III: el sistema FoodStore, una aplicación de gestión de pedidos de comida que permite administrar categorías, productos, usuarios y pedidos con sus líneas de detalle.

El sistema se compone de dos partes complementarias. La **Parte 1** es un frontend web desarrollado con TypeScript y Vite. La Parte 2 es un backend en Java con JPA, Hibernate y base de datos H2, que incorpora relaciones JPA, baja lógica, consultas JPQL personalizadas y un menú de consola navegable.

Como extensión del trabajo el backend incorpora además una API REST construida con Spring Boot, de modo que el frontend pueda consumir el sistema en línea (tal como se muestra en el video de referencia de la cátedra).

De esta forma, el proyecto cubre tanto el menú de consola exigido por la consigna como el sistema integrado de extremo a extremo.

Objetivos

- Integrar frontend y backend mediante una API REST.
- Aplicar persistencia de datos utilizando JPA, Hibernate y H2 Database.
- Implementar autenticación y diferenciación de roles.
- Permitir la gestión completa de categorías, productos y pedidos.
- Registrar pedidos con detalle de productos y actualizar el stock automáticamente.
- Mantener una interfaz clara y funcional para Cliente y Administrador.
- Documentar y publicar el proyecto en GitHub con instrucciones de ejecución.

Tecnologías utilizadas

Capa	Tecnologías y propósito
Frontend	TypeScript, Vite, HTML5, CSS3. Gestión de sesión mediante localStorage (autenticación y persistencia de la sesión del usuario).
Backend	Java 21, Spring Boot 3.3.5, Jakarta Persistence (JPA), Hibernate como proveedor ORM. Lombok para reducir código repetitivo

	Arquitectura en capas (Controller, Service, Repository, DTO y Entities).
Backend (API REST)	Spring Web para la exposición de servicios REST, Spring Data JPA para el acceso a datos, DTOs para la transferencia de información y CORS para permitir la comunicación entre el frontend y el backend.
Base de datos	H2 Database configurada en modo persistente mediante archivo (jdbc:h2:file:./data/foodstore), permitiendo conservar la información entre ejecuciones. El esquema de la base de datos es administrado automáticamente por Hibernate mediante spring.jpa.hibernate.ddl-auto=update.
Build y herramientas	Gradle (Kotlin DSL) como sistema de construcción del backend mediante Gradle Wrapper . npm para la gestión de dependencias y ejecución del frontend con Vite .
Control de versiones	Git para el control de versiones del proyecto y GitHub para el alojamiento de los repositorios del frontend y backend.

Recursos adicionales

Recurso	Enlace
Repositorio Frontend	https://github.com/MaruValletto/FoodStore-Frontend
Repositorio Backend	https://github.com/MaruValletto/FoodStore-Backend
Video demostración	https://youtu.be/ZtwxIQg_u5o

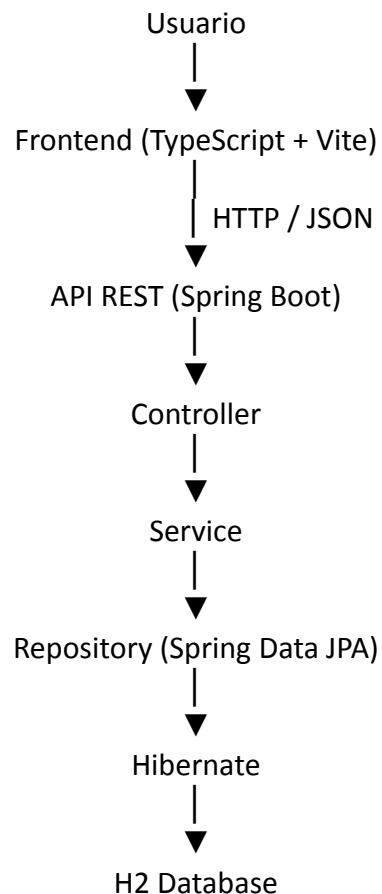
Arquitectura y flujo de funcionamiento

La aplicación fue desarrollada siguiendo una **arquitectura cliente-servidor**, compuesta por dos proyectos principales: un **frontend web**, encargado de la interacción con el usuario, y un **backend** desarrollado en Java con Spring Boot, responsable de la lógica de negocio y la persistencia de los datos.

La comunicación entre ambos componentes se realiza mediante **solicitudes HTTP** a una **API REST**, intercambiando información en formato **JSON**.

Flujo general de la aplicación

1. El usuario interactúa con la interfaz web desde el navegador.
2. El frontend envía una solicitud HTTP a la API REST del backend.
3. El **Controller** recibe la solicitud y la deriva al **Service** correspondiente.
4. El **Service** aplica las reglas de negocio y realiza las validaciones necesarias.
5. Cuando es necesario acceder a la información, el **Repository**, implementado con **Spring Data JPA**, interactúa con la base de datos **H2**.
6. El backend genera una respuesta en formato **JSON**, que es enviada nuevamente al frontend.
7. El frontend procesa la respuesta y actualiza la interfaz mostrada al usuario.



Frontend Web

Estructura y módulos

El frontend sigue la estructura propuesta por la cátedra (Vite + TypeScript) con páginas separadas por responsabilidad: autenticación (login y registro), tienda (catálogo, detalle de

producto y carrito), área de cliente (mis pedidos) y panel de administración (dashboard, categorías, productos y pedidos).

- **Autenticación y roles:** el usuario inicia sesión; sus datos se guardan en localStorage (sin la contraseña) y se redirige según el rol (ADMIN → panel de administración, USUARIO → catálogo). Cada página protegida verifica la sesión.
- **Catálogo:** grilla de productos con filtro por categoría, búsqueda en tiempo real y ordenamiento por nombre y precio. Solo se muestran productos con disponible = true y eliminado = false.
- **Detalle de producto:** selector de cantidad con validación de stock y botón de agregar al carrito.
- **Carrito:** se gestiona en localStorage; calcula subtotal, envío y total (el envío es una constante ENVIO = 0 documentada). El checkout solicita teléfono, dirección y forma de pago.
- **Mis pedidos:** historial del cliente con tarjetas por pedido (número, fecha, estado con color y total) y modal de detalle.
- **Administración:** dashboard con estadísticas, ABM de categorías y productos, y gestión de pedidos con cambio de estado.

Capa de integración (api.ts)

Siguiendo la indicación de la consigna —que el frontend debía aislar el acceso a datos para luego reemplazar los archivos JSON locales por la API REST— el proyecto centraliza todo el acceso al backend en un único módulo api.ts. Allí se define la URL base de la API y una función genérica de fetch tipada; las páginas importan funciones como getProductos(), login() o crearPedido() sin conocer detalles de red.

De este modo, la migración desde los JSON locales hacia la API consistió en cambiar la fuente de datos en un solo lugar: donde antes se hacía fetch('/data/productos.json') ahora se llama a la API (http://localhost:8080/api/productos). Este es exactamente el punto de integración que la consigna anticipaba.

Backend: modelo de dominio

El modelo se compone de una clase abstracta Base (@MappedSuperclass), la interfaz Calculable y cinco entidades JPA que extienden Base: Categoria, Producto, Usuario, Pedido y DetallePedido.

Entidades

Entidad	Campos propios
Base (abstracta)	id (Long, autogenerado), eliminado (boolean, default false), createdAt (LocalDateTime).
Calculable (interfaz)	Define calcularTotal(): void, implementado por Pedido.
Categoria	nombre, descripcion, imagen.
Producto	nombre, precio, descripcion, stock, imagen, disponible.
Usuario	nombre, apellido, mail (único), celular, contrasena, rol (enum Rol).
Pedido	fecha (LocalDate), estado (enum), total, formaPago (enum), dirección y teléfono. Implementa Calculable.
DetallePedido	cantidad, subtotal, y el Producto referenciado (@ManyToOne).

Relaciones entre entidades

Relación	Cardinalidad	Descripción
Categoria → Producto	1 — N	Una categoría agrupa muchos productos.
Usuario → Pedido	1 — N	Un usuario puede tener muchos pedidos.
Pedido → DetallePedido	1 — N	Composición: los detalles se crean y gestionan desde el pedido (addDetallePedido).

DetallePedido → Producto	N — 1	Cada línea referencia el producto que representa (@ManyToOne).
---------------------------------	-------	--

Enums

Enum	Valores
Rol	ADMIN, USUARIO
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO

Backend — Modo consola (JPA puro)

La primera etapa del backend fue desarrollada utilizando **JPA puro**, implementando un menú de consola como requería la consigna. Este modo se encuentra organizado mediante una capa de utilidades y repositorios propios.

- **JPAUtil:** administra una única instancia de EntityManagerFactory durante toda la ejecución de la aplicación.
- **BaseRepository<T>:** implementa un CRUD genérico (guardar, buscarPorId, listarActivos y eliminarLogico). Cada operación administra su propio EntityManager, controlando las transacciones mediante begin, commit y rollback cuando corresponde.
- **Repositorios específicos:** implementan consultas JPQL tipadas con parámetros nombrados, permitiendo buscar productos por categoría, usuarios por mail, pedidos por usuario y pedidos por estado, filtrando siempre los registros activos (eliminado = false).
- **Menú de consola:** permite administrar Categorías, Productos, Usuarios y Pedidos, además de generar reportes básicos.

Alta de pedido (transacción atómica)

El alta de pedidos constituye la operación de negocio más importante del sistema.

Durante la creación de un pedido se realizan las siguientes acciones dentro de una única transacción:

- Validación de existencia del producto.
- Validación de disponibilidad.
- Validación de stock suficiente.
- Creación del Pedido.
- Creación de los DetallePedido.
- Cálculo del subtotal de cada detalle.
- Descuento del stock correspondiente.
- Cálculo del total mediante `calcularTotal()`.
- Persistencia completa del pedido.

En caso de producirse cualquier error durante el proceso, la transacción es revertida completamente mediante **rollback**, garantizando la consistencia de los datos.

Backend — API REST (Spring Boot)

Sobre el mismo modelo de dominio desarrollado en JPA se construyó una **API REST** utilizando **Spring Boot**, organizada mediante las capas **Controller**, **Service** y **Repository**.

Los controladores devuelven **DTOs** en lugar de las entidades JPA, evitando exponer el modelo interno y desacoplando la API de la estructura de persistencia.

La aplicación incorpora además un **DataInitializer**, responsable de cargar automáticamente los datos iniciales del sistema la primera vez que se ejecuta.

Método	Ruta	Descripción
GET	/api/categorias	Lista las categorías activas.
GET	/api/categorias/{id}	Obtiene una categoría por ID.
POST	/api/categorias	Crea una nueva categoría.
PUT	/api/categorias/{id}	Modifica una categoría existente.
DELETE	/api/categorias/{id}	Realiza la baja lógica de una categoría.
GET	/api/productos	Lista productos activos.
GET	/api/productos?categorial d={id}	Filtra productos por categoría.
GET	/api/productos/{id}	Obtiene un producto por ID.
POST / PUT / DELETE	/api/productos	Alta, modificación y baja lógica de productos.
GET	/api/usuarios	Lista usuarios activos.
POST	/api/usuarios/login	Valida credenciales e inicia sesión.
POST	/api/usuarios/registro	Registra un nuevo usuario cliente.
GET	/api/pedidos	Lista pedidos.
GET	/api/pedidos?estado=...	Filtra pedidos por estado.
POST	/api/pedidos	Registra un nuevo pedido con validación de stock.
PATCH	/api/pedidos/{id}/estado	Actualiza el estado de un pedido.

CORS: el backend configura CORS para permitir la comunicación entre el frontend desarrollado con **Vite** y la API REST implementada con **Spring Boot** durante el desarrollo.

Decisiones técnicas

- **Transacciones atómicas:** la creación de pedidos se ejecuta dentro de una única transacción, garantizando que el pedido y el descuento de stock se confirmen o se cancelen conjuntamente.
- **Consultas JPQL:** las búsquedas específicas se implementan mediante consultas JPQL tipadas utilizando parámetros nombrados.
- **DTOs:** la API REST utiliza DTOs para desacoplar el modelo de persistencia del contrato expuesto al frontend y evitar problemas de serialización.
- **Baja lógica:** las entidades no se eliminan físicamente; simplemente se marcan como eliminadas, preservando el historial.
- **Capa de acceso a la API:** el frontend centraliza todas las solicitudes HTTP en el módulo api.ts, facilitando el mantenimiento y la integración con el backend.
- **Persistencia mediante H2:** la base de datos fue configurada en modo persistente (jdbc:h2:file:./data/foodstore), permitiendo conservar la información entre distintas ejecuciones de la aplicación.
- **Arquitectura cliente-servidor:** el proyecto reutiliza el modelo de dominio desarrollado inicialmente en JPA y lo amplía mediante Spring Boot para construir una aplicación web completamente integrada.

Datos de prueba y credenciales

El sistema incorpora un conjunto de datos iniciales cargados automáticamente mediante **DataInitializer**, permitiendo utilizar la aplicación sin necesidad de realizar una carga manual.

Recurso	Datos iniciales
Categorías	6 categorías (Pizzas, Hamburguesas, Bebidas, Postres, Empanadas y Ensaladas).
Productos	Catálogo inicial distribuido entre las distintas categorías, incluyendo precio, stock, disponibilidad e imagen.
Usuarios	Un usuario Administrador y un usuario Cliente de prueba.

Pedidos	Los pedidos son generados desde el flujo de compra del cliente y permanecen almacenados en la base de datos H2 persistente.
----------------	---

Credenciales de prueba

Rol	Email	Contraseña
ADMIN	admin@foodstore.com	123456
USUARIO	juan@mail.com	pass123

Con el objetivo de verificar el correcto funcionamiento del sistema, se realizaron las siguientes pruebas funcionales:

- Registro de un nuevo usuario y posterior inicio de sesión.
- Intento de inicio de sesión con credenciales incorrectas, verificando la validación correspondiente.
- Navegación del catálogo utilizando búsqueda, filtros por categoría y ordenamiento de productos.
- Agregado de productos al carrito y modificación de cantidades.
- Ejecución completa del proceso de checkout con generación de un nuevo pedido.
- Verificación del descuento automático de stock luego de confirmar una compra.
- Consulta del pedido generado desde la sección **Mis Pedidos** del cliente.
- Cambio del estado de un pedido desde el panel del Administrador.
- Verificación de la actualización del estado del pedido desde la vista del cliente.
- Alta, modificación y baja lógica de categorías.
- Alta, modificación y baja lógica de productos.
- Reinicio del backend verificando la persistencia de imágenes, usuarios, productos, categorías y pedidos.
- Verificación de la información almacenada mediante la consola **H2 Console**.

Reglas de negocio implementadas

Durante el desarrollo del sistema se implementaron las siguientes reglas de negocio:

- Validación de stock antes de permitir la creación de un pedido.
- Descuento automático del stock a confirmar una compra desde el proceso de checkout.
- Implementación de **baja lógica** para categorías, productos y demás entidades, preservando la información histórica.
- Actualización del estado de los pedidos exclusivamente por parte del usuario con rol **Administrador**.
- Visualización por parte del cliente del estado actualizado de sus pedidos desde la sección **Mis Pedidos**.
- Persistencia de la información mediante **H2 Database** configurada en modo archivo, conservando los datos entre distintas ejecuciones de la aplicación.

Cobertura de Historias de Usuario

Backend

Historia	Implementación
HU-01 a HU-04	CRUD genérico mediante BaseRepository y consultas JPQL específicas.
HU-05 a HU-07	Gestión completa de Categorías con validaciones y baja lógica.
HU-08 a HU-11	Gestión completa de Productos con validaciones, stock y filtrado por categoría.
HU-12 a HU-15	Gestión de Usuarios con validación de mail único e inicio de sesión.
HU-16 a HU-19	Alta transaccional de pedidos, cambio de estado y actualización automática de stock.
HU-20 / HU-21	Consultas y reportes implementados sobre el modelo de pedidos.

Frontend

Historia	Implementación
FHU-01 / FHU-02	Inicio de sesión y registro de usuarios con persistencia de sesión.
FHU-03 / FHU-04	Catálogo con búsqueda, filtros y ordenamiento de productos.
FHU-05 / FHU-06	Carrito de compras, checkout e historial de pedidos del cliente.
FHU-07 a FHU-10	Dashboard administrativo, gestión de Categorías, Productos y Pedidos mediante integración con la API REST.

Instrucciones de ejecución**Backend — API REST (Spring Boot)**

Desde la carpeta del backend ejecutar:

```
./gradlew bootRun
```

En Windows:

```
gradlew.bat bootRun
```

La API quedará disponible en:

```
http://localhost:8080/api
```

La consola de administración de H2 puede accederse desde:

```
http://localhost:8080/h2-console
```

Backend — Menú de consola (JPA)

La versión de consola puede ejecutarse desde el entorno de desarrollo (IntelliJ IDEA) ejecutando la clase **Main**.

El menú permite administrar:

- Categorías
- Productos
- Usuarios
- Pedidos
- Reportes

Frontend

Desde la carpeta del frontend, ejecutar en la terminal:

```
npm install
```

```
npm run dev
```

La aplicación quedará disponible en la dirección indicada por Vite (por defecto):

```
http://localhost:5173
```

Ejecución integrada

Para ejecutar el sistema completo:

1. Iniciar el backend mediante: `./gradlew bootRun`
2. Luego iniciar el frontend mediante: `npm run dev`

De esta forma el frontend consumirá la API REST desarrollada en Spring Boot y ambos proyectos funcionarán de manera integrada.

Conclusión

El proyecto **FoodStore** cumple los objetivos planteados para el Trabajo Práctico Integrador, implementando una solución completa basada en una arquitectura cliente-servidor.

El backend fue desarrollado utilizando **Spring Boot**, **Spring Data JPA**, **Hibernate** y **H2 Database**, incorporando un modelo de dominio persistente, baja lógica, transacciones, consultas mediante JPQL y una API REST para la gestión de usuarios, categorías, productos y pedidos.

El frontend, desarrollado con **HTML**, **CSS**, **TypeScript** y **Vite**, permite a los usuarios interactuar con el sistema mediante una interfaz intuitiva que incluye autenticación, catálogo de productos, carrito de compras, gestión de pedidos y un panel administrativo para la administración integral del negocio.

La integración entre ambos proyectos demuestra el recorrido completo de la información, desde la interacción del usuario hasta la persistencia de los datos, aplicando conceptos de arquitectura en capas, separación de responsabilidades e intercambio de información mediante servicios REST.

Como resultado, se obtuvo una aplicación funcional, escalable y mantenible, que integra los conocimientos adquiridos durante la materia y constituye una base sólida para futuras ampliaciones.